

Module 0 — Foundations & Reproducibility: Runbook

This is an **operational** document — step-by-step execution instructions, expected outputs, and troubleshooting. For the conceptual "why," see this module's [README.md](#). For the full curriculum context, see [docs/curriculum/modular_bioinformatics_curriculum.md](#).

Prerequisites

- Access to an HPC cluster with Slurm (sbatch, sacct, seff, module all on PATH), or an equivalent environment.
- Conda or Mamba installed (for scripts 7 and 10's environment steps). Check with command `-v mamba / command -v conda`; if neither exists, run `module avail` and load a Miniconda/Miniforge module first.
- Git installed (command `-v git`). Nearly always already available; if not, `module load git` or install it via script 7's environment.
- A scratch/working directory with at least ~30GB free (PRJNA1518998's two reference SRR runs are ~3GB each; QC/trimming intermediates roughly double that; leave headroom for the module's own storage-safety checks).

Execution order

Run every step from the module's `scripts/` directory unless noted.

Step	Command	Where	Expected result
1	<code>bash 1_The_Hardcoded_Script_SingleRun.sh</code> (edit the hardcoded SRR first)	login node or sbatch	One sample downloaded, QC'd, trimmed, QC'd again
2	Create <code>srr_list.txt</code> (one accession per line)	login node	Plain text file, no header, no blank lines
3	<code>sbatch 2_Automated_Script- _Processes_ALLRuns.sh</code>	compute	All samples in <code>srr_list.txt</code> processed; no storage guard yet
4	<code>sbatch 3_Automated_Script_Hardcoded_20GB_Limit.sh</code>	compute	Same as step 3, loop halts if usage exceeds a hardcoded 20GB
5	<code>sbatch 4_Automated_Script_User- DefinedMaximumSpace.sh</code>	compute	Same, but the limit is a labeled variable at the top of the file
6	<code>sbatch 5_Automated_Script_System- BoundDynamicLimit.sh</code>	compute	Loop halts based on the filesystem's <i>actual</i> free space, not a fixed number
7	<code>sbatch 6_Predicting_Memory_Requirements.sh</code>	compute	Same pipeline, plus a <code>seff</code> -based entry appended to <code>mem_usage_log.txt</code>
8	<code>bash suggest_mem_from_history.sh</code> <code>qc_mem_predict 25</code>	login node	Printed <code>--mem</code> recommendation for the <i>next</i> submission (requires step 7 run at least once)
9	<code>bash 7_Environment_Reproducibility.sh</code> <code>create</code>	login node	A new Conda/Mamba environment <code>mibo8110-module0</code> , built from <code>environment.yml</code>
10	<code>bash 7_Environment_Reproducibility.sh</code> <code>verify</code>	login node	Confirms the active environment's key tool versions match <code>environment.yml</code>
11	<code>bash 8_Git_Reproducibility_Workflow.sh</code> <code>init</code>	login node	New Git repository + a data-safe <code>.gitignore</code> in the current directory

Step	Command	Where	Expected result
12	<code>bash 8_Git_Reproducibility_Workflow.sh snapshot</code>	login node	Commits scripts + <code>environment.yml</code> ; prints the new commit hash
13	<code>bash 9_File_Format_Validation.sh <any directory></code>	login node	Per-file PASS/WARN/FAIL table for every recognized format found
14	<code>sbatch 10_Integrated_Foundations_Capstone.sh</code>	compute	The full integrated run: storage guard + memory log + environment fingerprint + Git stamp + output format validation + <code>REPRODUCIBILITY_REFLECTION.md</code>
15	Fill in the "Questions to answer by hand" section of <code>REPRODUCIBILITY_REFLECTION.md</code>	login node	Completed portfolio artifact, ready to commit (<code>bash 8_Git_Reproducibility_Workflow.sh snapshot</code>)

Steps 1–8 can be run and understood independently, in order, as the storage/memory progression. Steps 9–14 depend on steps 1–8 only in that they reuse the same working directory conventions and `srr_list.txt` — they do not require you to have run every earlier script's `sbatch` job first, **except** step 14, which assumes an `environment.yml` (step 9) and an initialized Git repository (step 11) already exist at the expected relative paths.

Expected outputs, by step

- **Steps 1–8:** `raw_reads/`, `qc_before/`, `trimmed_reads/`, `qc_after/`, `fastp_reports/`, `multiqc_raw_final.html`, `multiqc_trimmed_final.html`, `failed_downloads.txt` (only if any download failed), `mem_usage_log.txt` (from step 7 onward).
- **Step 9:** `environment.yml` in the current directory; a new named Conda environment (check with `conda env list`).
- **Steps 11–12:** `.git/` directory, `.gitignore`, and one commit per snapshot call with staged changes.
- **Step 13:** stdout table only (no files written) — a PASS/WARN/FAIL line per recognized file, plus a summary count.
- **Step 14:** everything from steps 1–8's pipeline, under `/scratch/${whoami}/capstone/module0/`, plus `logs/run_<timestamp>.log` (the full provenance-stamped run log) and `REPRODUCIBILITY_REFLECTION.md` in that same working directory.

Troubleshooting

Symptom	Likely cause	Fix
ERROR: <code>srr_list.txt</code> not found or empty	File missing, wrong directory, or truly empty	Create it in the same directory as the script; confirm with <code>wc -l srr_list.txt</code>
Loop appears to run zero iterations	<code>srr_list.txt</code> has Windows line endings (<code>\r\n</code>) or was read with <code>for X in \$(cat file)</code> in a modified copy	Use the <code>while IFS= read -r</code> pattern already in scripts 2–6/10; strip <code>\r</code> with <code>tr -d '\r' < file > file.clean</code> if needed
integer expression expected from a <code>du/df</code> -based comparison	Unexpected <code>du/df</code> output format on this system	Scripts 3–6/10 already numeric-sanity-check this and print a WARNING instead of crashing; if you see the raw bash error, you are likely running an unmodified/older copy
ERROR: neither 'mamba' nor 'conda' was found (script 7)	No Conda/Mamba on PATH	<code>module avail</code> and load a Miniconda/Miniforge module, or install Miniforge in your home directory

Symptom	Likely cause	Fix
bash 7_Environment_Reproducibility.sh verify warns the active env doesn't match	You activated a different/older environment, or environment.yml was edited after the environment was built	Re-run create, or <code>conda env update -n mibo8110-module0 -f environment.yml</code>
fatal: pathspec '...' did not match any files from git add	Only relevant if you hand-modify script 8 — the shipped version uses <code>git add -A .</code> , which does not have this failure mode	Use <code>git add -A .</code> rather than listing specific extensions, so a missing file type doesn't abort the whole add
8_Git_Reproducibility_Workflow.sh stamp prints <code>git_dirty=true</code>	Files were changed after the last snapshot	Run <code>snapshot</code> again before trusting the commit hash for a result you plan to report
Script 9 reports <code>samtools not on PATH -- skipped</code> for a BAM file	samtools module/environment not loaded in the current shell	<code>module load SAMtools/1.16.1-GCC-11.3.0</code> , or activate the script 7 environment
Script 9 reports a FAIL on a FASTQ pair's read counts	One mate file is truncated, or <code>_1/_2</code> files were mismatched between different samples	Re-download the affected accession; confirm <code>_1/_2</code> (or <code>_R1/_R2</code>) filenames actually belong to the same sample
sacct/seff return nothing for suggest_mem_from_history.sh	No completed job with that name yet (the documented "bootstrap problem")	Run <code>6_Predicting_Memory_Requirements.sh</code> (or the step-14 capstone) at least once first, then re-run the helper
Step 14 prints <code>git_commit=UNAVAILABLE</code> in its log/reflection	Relative path to <code>8_Git_Reproducibility_Workflow.sh</code> doesn't resolve from the compute job's working directory	Confirm you are using this module's standard <code>/scratch/\$whoami/capstone/module0/</code> layout, three directories below <code>scripts/</code> , or edit the path constants at the top of script 10

Completion checklist (portfolio artifact)

The curriculum's Module 0 assessment asks for: *"a repository containing a small pipeline, environment specification, README, input checks, log, outputs, and a reproducibility reflection."* Confirm each is present before considering the module complete:

- ☐ **Pipeline** — at minimum, one successful run of `10_Integrated_Foundations_Capstone.sh` (or the equivalent scripts 1–6 progression) against `srr_list.txt`.
- ☐ **Environment specification** — `environment.yml` exists (script 7) and `bash 7_Environment_Reproducibility.sh verify` reports no mismatch.
- ☐ **README** — this module's `README.md` is present and describes what each script does (already provided; extend it if you add scripts).
- ☐ **Input checks** — `srr_list.txt` existence/non-empty checks (built into every numbered script) and `9_File_Format_Validation.sh` run against at least the raw downloaded files.
- ☐ **Log** — a run log exists (`logs/run_<timestamp>.log` from step 14, or the printed stdout captured from any earlier step) showing what actually executed.
- ☐ **Outputs** — `multiqc_raw_final.html`, `multiqc_trimmed_final.html`, and the trimmed FASTQ files are present and pass `9_File_Format_Validation.sh`.
- ☐ **Reproducibility reflection** — `REPRODUCIBILITY_REFLECTION.md` exists and its "Questions to answer by hand" section is filled in, not left blank.
- ☐ **Responsible practice** — the Responsible Practice Checklist in `README.md` is checked off (dataset license, no PII, no raw data committed to Git).

- [] **Version control** — `git log` in the repository shows at least one commit (from `8_Git_Reproducibility_Workflow.sh` snapshot) covering the final state of the scripts and `environment.yml` used to produce the submitted outputs.